

Week 10 Lecture

Welcome! Today we will look at the concepts of:

- Easing in Animation
 - Transformations (Translate, Rotate, Scale)
-

Easing

Easing is a technique for smoothing out animations to look more natural. It describes how something starts and stops. There are lots of types of easing, and we will only cover an example today.

We can apply easing to all kinds of different values that change over time, including position, size, and color.

You can have a look at all the types of easings here: easings.net  (<https://easings.net/>).

Linear Interpolation (LERP)

One of the ways in which we can achieve easing is through the `lerp()` command: it lets us linearly interpolate between two values. We provide a start value, a stop value, and the amount to interpolate between.

For example, in p5, if I write:

```
print(lerp(10,20,0.5))
```

the value returned is:

```
15
```

since 15 is mid way between 10 and 20. But, if I write:

```
print(lerp(10,20,0.25))
```

the value returned is:

```
12.5
```

since the gap between $20 - 10 = 10$, and $10 / 4 = 2.5$, so $10 + 2.5 = 12.5$.

Let's see how we can use the lerp idea in an easing example: Let's try and draw a circle and have the circle follow around our mouse:

```
x = 200;
y = 200;

function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
  fill(255,0,0)

  x = lerp(x, mouseX, 0.05);
  y = lerp(y, mouseY, 0.05);

  circle(x, y, 30);
}
```

This will cause the x and y values to be continuously updated based on where our Mouse is, and consequently will generate the animation.

Transformations

Transformations enable you to change the sizes, positions and orientations of objects:

- Translate (move something from one point to another)
- Rotate (rotate something by a given number of angles around a point)
- Scale (stretch or shrink something by a certain value)

The origin point

The origin point is the point around which all transformations (translate, rotate and scale) are applied. Understanding the origin point is essential to understanding transformations.

The origin point is the coordinate (0, 0). Which starts in the top left hand corner of our canvas. But, the origin point can be moved with `translate()`.

Translate

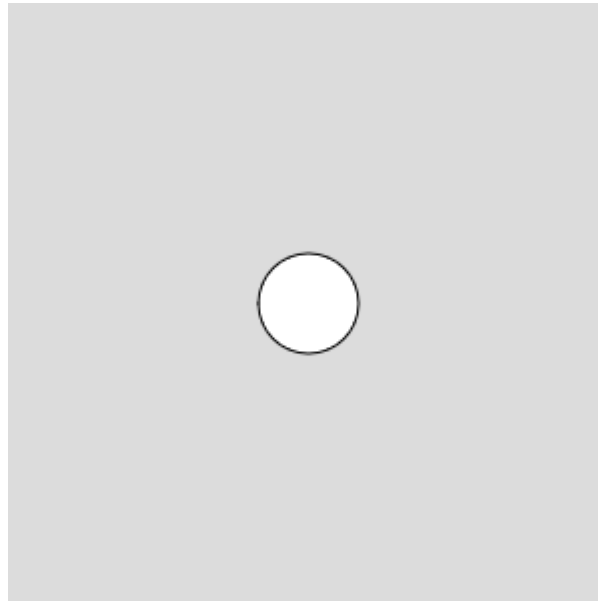
Normally the origin point is in the top left corner of the canvas, but with `translate()` we can move it anywhere we want.

All shapes drawn after `translate(x, y)` will be relative to the new coordinate `(x, y)`.

ketch.js

```
function setup() {  
  createCanvas(300, 300)  
}  
  
function draw() {  
  background(220)  
  translate(width / 2, height / 2);  
  circle(0, 0, 50)  
}
```

Preview



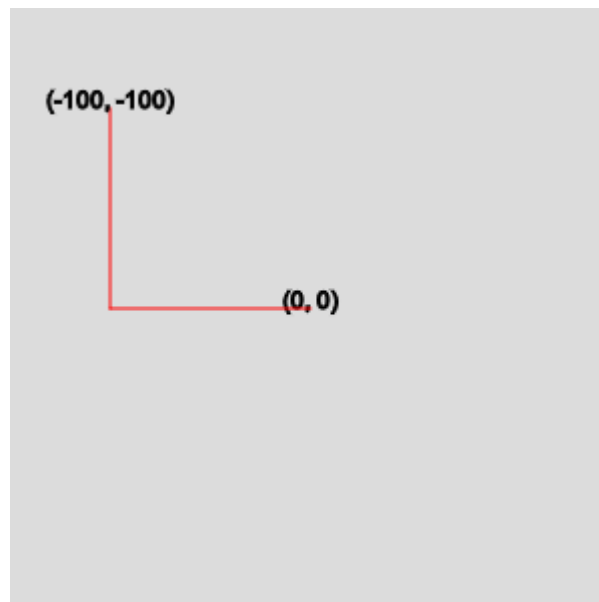
Notice that the shape is drawn at (0,0) yet it appears in the center of the canvas. This is because the origin point has been translated to the center of the canvas.

Because coordinates are relative to the origin point, it's possible that negative coordinates might be on the visible area of the canvas.

ketch.js

```
function setup() {  
  createCanvas(300, 300)  
  textAlign(CENTER)  
}  
  
function draw() {  
  background(220)  
  translate(width / 2, height / 2);  
  text("(0, 0)", 0, 0)  
  
  stroke('red')  
  line(0, 0, -100, 0)  
  line(-100, 0, -100, -100)  
  
  stroke('black')  
  text("(-100, -100)", -100, -100)  
}
```

Preview



Sequential calls to `translate()` will be relative to the new origin point which was set by the previous `translate()` call. The translations accumulate.

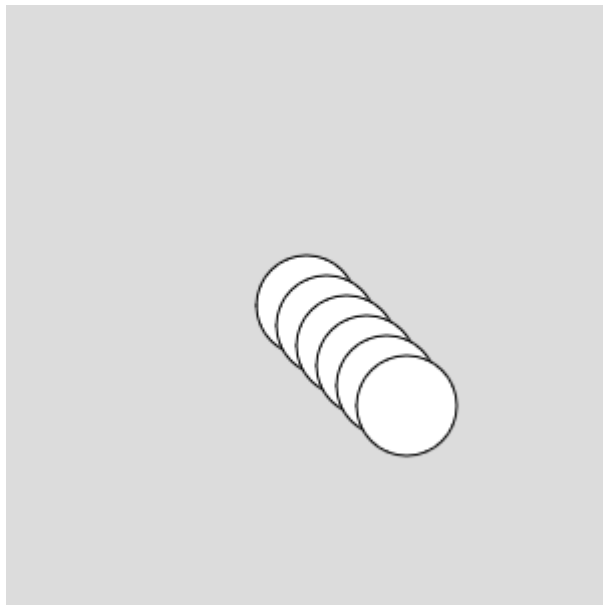
ketch.js

```
function setup() {  
  createCanvas(300, 300)  
}
```

Preview

```
function draw() {
  background(220)
  translate(width / 2, height / 2);
  circle(0, 0, 50)

  for (let i = 0; i < 5; i++) {
    translate(10, 10)
    ellipse(0, 0, 50)
  }
}
```



Push and Pop

If we want to store the current transformations (and styling) we use `push()`.

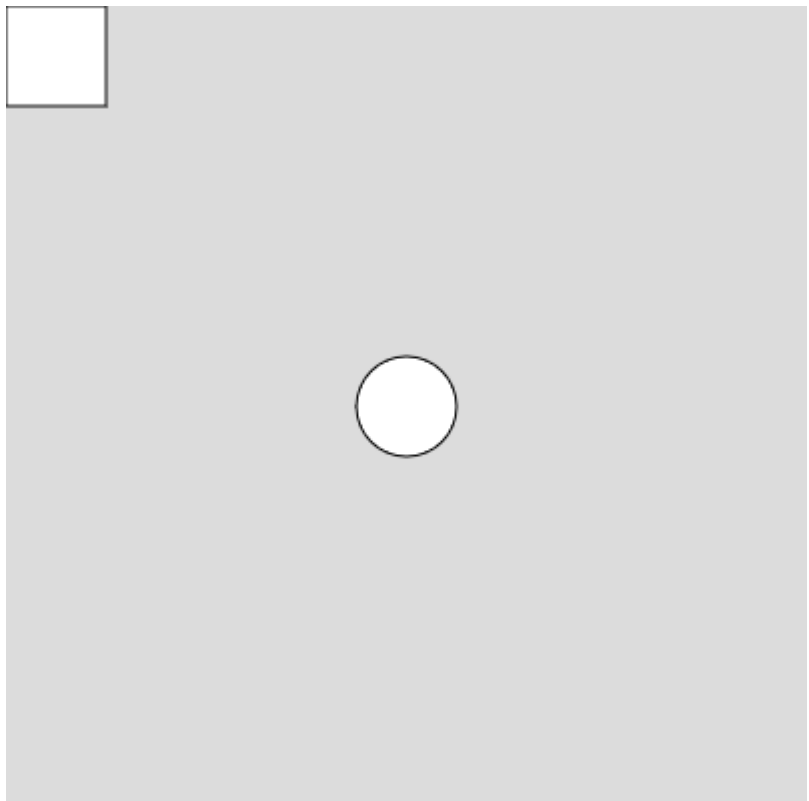
When we want to restore the 'pushed' transformations (and styling) we use `pop()`.

ketch.js

```
function setup() {
  createCanvas(400, 400)
}

function draw() {
  background(220)
  push()
    translate(width / 2, height / 2)
    circle(0, 0, 50)
  pop()
  square(0, 0, 50)
}
```

Preview



As stated in the previous slide `push()` and `pop()` also store and revert styling functions.

These include but are not limited to: `fill()`, `noFill()`, `noStroke()`, `stroke()`, `strokeWeight()`, `strokeCap()`, `strokeJoin()`, `rectMode()`, `ellipseMode()` and `colorMode()`.

See the p5 reference page for `push()` ➔ (<https://p5js.org/reference/#/p5/push>) to see the entire list.

It's possible to `push()` multiple layers of transformations.

When you `pop()`, you revert to the last 'pushed' state. That means that you undo all transformations you have made since the previous `push()`.

ketch.js

```
function setup() {
  createCanvas(400, 400)
}

function draw() {
  background(220)

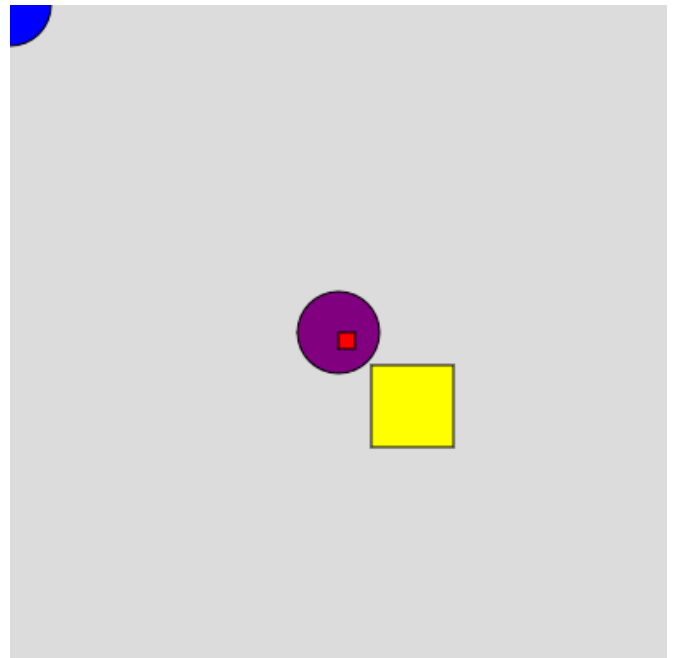
  push()
  fill('purple')
  translate(width / 2, height / 2)
  circle(0, 0, 50)

  push()
  fill('yellow')
  translate(20, 20)
  square(0, 0, 50)
  pop()

  fill('red')
  square(0, 0, 10)
  pop()

  fill('blue')
  circle(0, 0, 50)
}
```

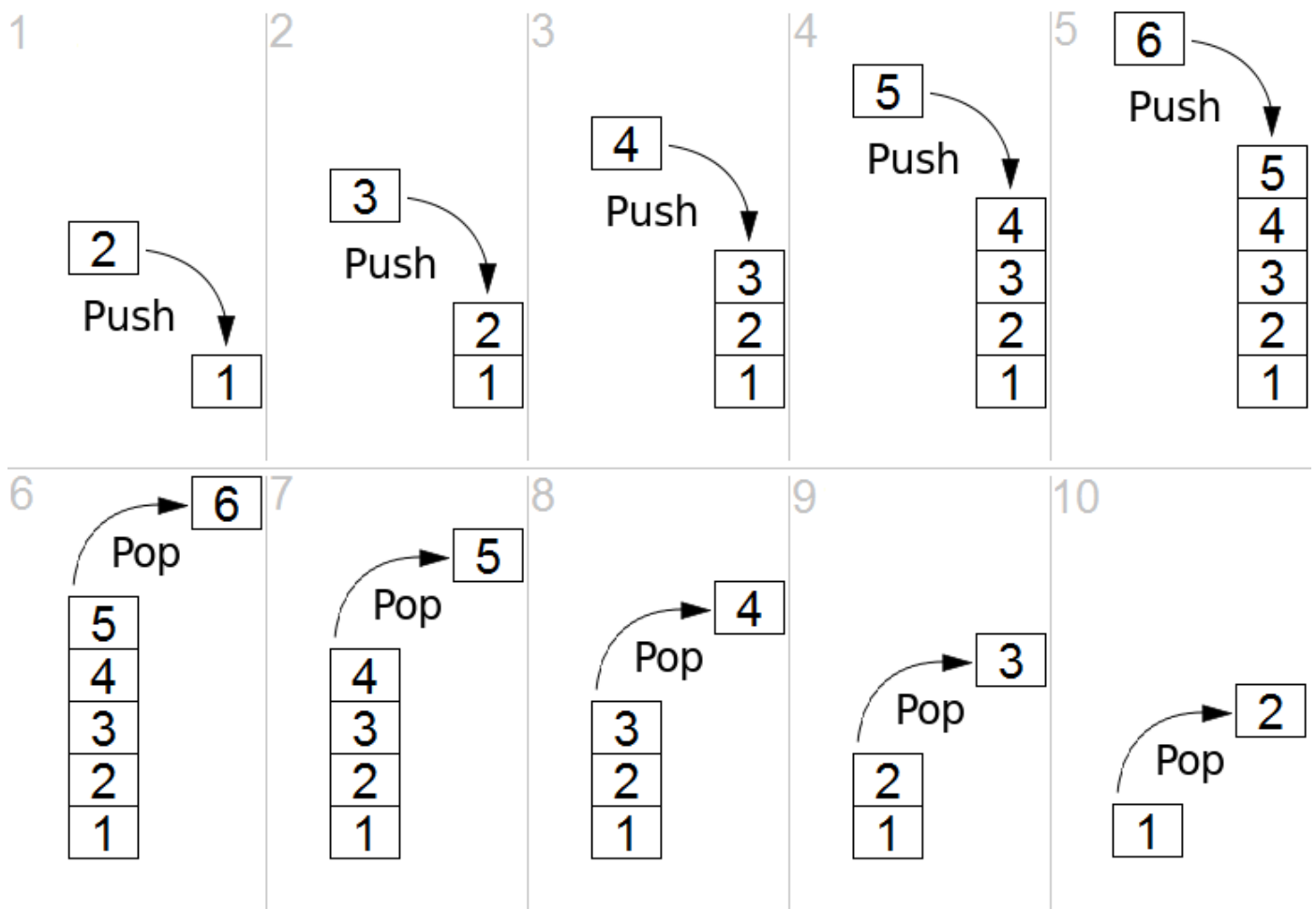
Preview



Did you pay attention to the order each shape was drawn?

Push Pop and Stacks

`push()` and `pop()` store and revert transformation states based on a LIFO (last in-first out) order. This datastructure is commonly called a stack.



Stack illustration from: https://computersciencewiki.org/images/b/b4/Lifo_stack.png 
https://computersciencewiki.org/images/b/b4/Lifo_stack.png

Rotate

Rotate allows us to rotate a shape around a point by a given number of angles or radians.

```
let angle = 0;
let width = 400;
let height = 400;

function setup() {
  createCanvas(width,height);
  angleMode(DEGREES);
}

function draw() {
  background(200);

  push();
  translate(width/2, height/2);

  angle += 1;
  rotate(angle);

  noStroke();
  fill(200,0,0);
  rectMode(CENTER);
  square(0,0,150);
  pop();
}
```

Let's look at how this behaves.

Notice that the `rotate()` transformations accumulate just like `translate()`. In fact, all transformations accumulate with themselves and each other. This allows us to build complex patterns.

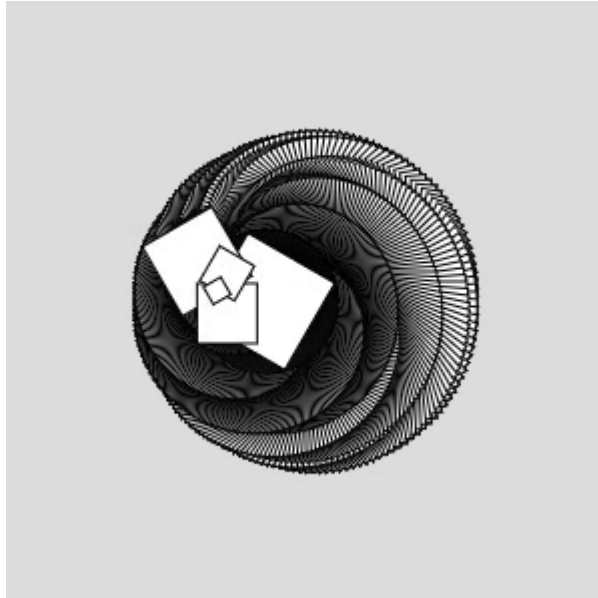
ketch.js

```
let rot = 0

function setup() {
  createCanvas(300, 300)
  angleMode(DEGREES)
  rectMode(CENTER)
  background(220)
}

function draw() {
  translate(80, 150)
  for (let i = 0; i < 5; i++) {
    let size = map(i, 0, 4, 50, 10)
    translate(size, 0)
    rotate(frameCount)
    square(0, 0, size)
  }
}
```

Preview



Here the map function works as follows: <https://p5js.org/reference/p5/map> 
(<https://p5js.org/reference/p5/map>)

map()

Re-maps a number from one range to another.

For example, calling `map(2, 0, 10, 0, 100)` returns 20. The first three arguments set the original value to 2 and the original range from 0 to 10. The last two arguments set the target range from 0 to 100. 20's position in the target range [0, 100] is proportional to 2's position in the original range [0, 10].

Thus, in the example above, `i` (the index of the iteration) is originally going from 0 to 4 (as per our for loop), but is newly mapped between 50 and 10. This gives us a size relative to the iteration number - the first square is the smallest, and the last one the biggest.

Note that each square is moving on its own, but the whole appears as a combined system.

Scale

`scale()` stretches or shrinks shapes. Scale makes coordinates closer together or further apart.

ketch.js

```
function setup() {
  createCanvas(300, 300)
```

Preview

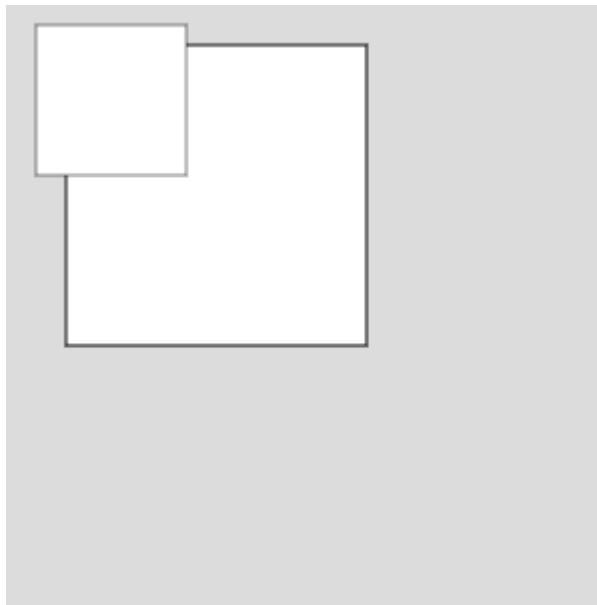
```

}

function draw() {
  background(220);
  square(30, 20, 150);

  // Shrink the coordinates
  scale(0.5);
  square(30, 20, 150);
}

```



ketch.js

```

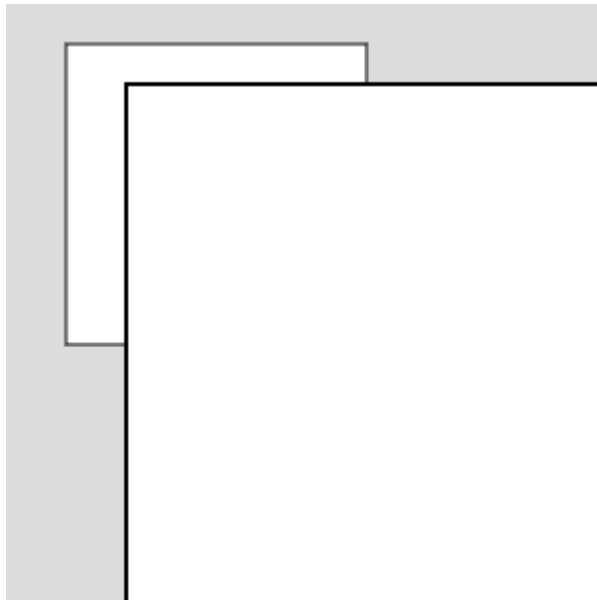
function setup() {
  createCanvas(300, 300)
}

function draw() {
  background(220);
  square(30, 20, 150);

  // Expand the coordinates
  scale(2);
  square(30, 20, 150);
}

```

Preview

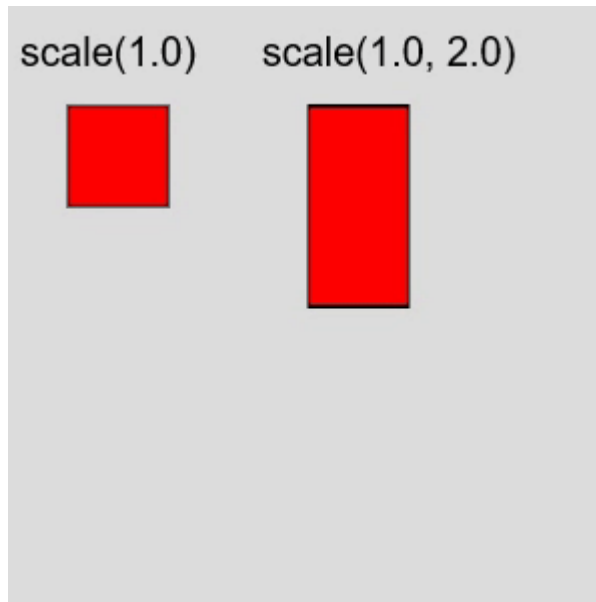


Notice that scale also affects the `strokeWeight()` of the strokes.

`scale()` can take 1 or 2 parameters when working in a 2D context, and 3 if you are working in 3D. Providing 1 parameter will proportionally scale the coordinate system along both axes. Providing 2 parameters will disproportionately scale along the x and y axes.

Illustration

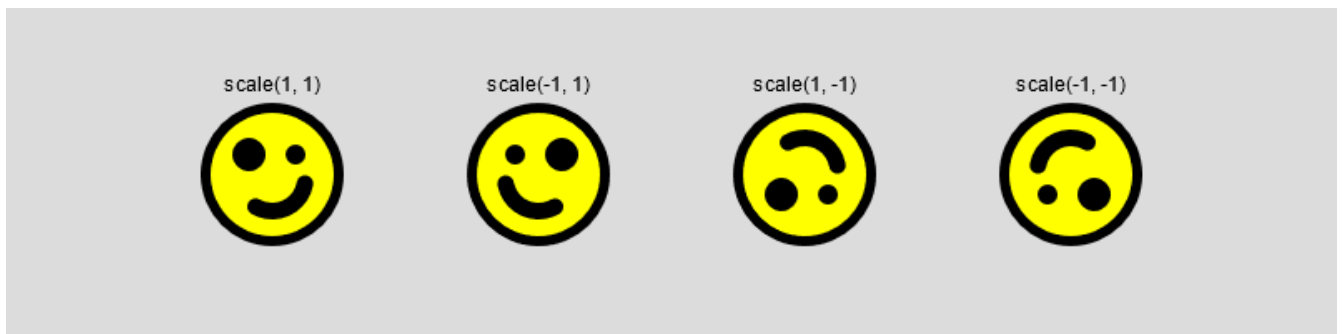
sketch.js



We can also use `scale()` to invert or flip the canvas coordinates.

Illustration

[sketch.js](#)



ketch.js

Preview

```
let smileyScale = 8

function setup() {
  createCanvas(400, 400)
}

function draw() {
  background(220)
  translate(width / 2, height / 2)
  let flip = sin(frameCount) * smileyScale //[-1,1]
  scale(flip, smileyScale)
  smiley()
}

function smiley() {
  fill("yellow")
  strokeWeight(3)
  ellipse(0, 0, 40)
```

```

fill("black")
noStroke()
ellipse(-7, -6, 10)
ellipse(7, -6, 6)
noFill()
stroke("black")
strokeWeight(5)
angleMode(DEGREES)
arc(0, 1, 20, 20, 10, 120)
}

```



Notes on transformations

- The origin point is the centre of transformations.
- The coordinates move: We aren't directly - - manipulating shapes, we are moving the canvas coordinate system.
Transformations add up: All transformations are accumulative.
- The order of transformations is important: The resulting transformation state will change depending on the order that transformations are applied.
E.g. `rotate()` then `translate()` will be different to `translate()` then `rotate()`.
- Transformations reset: At the end of each `draw()` call the transformation state resets to default.

Feel free to look at these examples for additional content to help understand transformations in p5js:

- **Transforming the Coordinate System** ➞ <https://editor.p5js.org/DWAIL/sketches/E8Tc4mSr1>
- **Angles when scaled disproportionately** ➞ <https://editor.p5js.org/DWAIL/sketches/YAbODHwaP>
- **Spirograph Harmony** ➞ <https://editor.p5js.org/DWAIL/sketches/3c30yJpfC>

References and Notes

Philip, G. (2003). What is generative art? Complexity theory as a context for art theory.

http://philipgalanter.com/downloads/ga2003_paper.pdf 

(http://philipgalanter.com/downloads/ga2003_paper.pdf) .

For an entertaining introduction to information theory, see: **3Blue1Brown Solving Wordle using information theory**  (<https://www.3blue1brown.com/lessons/wordle>) .

If you want an introduction to the mathematics behind these techniques, I recommend these three videos on topics in linear algebra:

- <https://www.3blue1brown.com/lessons/vectors> 
(<https://www.3blue1brown.com/lessons/vectors>)
- <https://www.3blue1brown.com/lessons/span> 
(<https://www.3blue1brown.com/lessons/span>)
- <https://www.3blue1brown.com/lessons/linear-transformations> 
(<https://www.3blue1brown.com/lessons/linear-transformations>)

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.

[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html).